

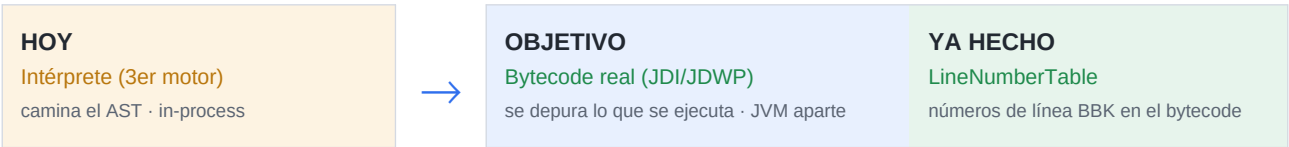
BoxBreaker — Debugger sobre bytecode real

Proyecto rpg-bbk-compiler | Análisis: acoplar el debugger al programa compilado | 20 de junio de 2026

Resumen

Hoy el debugger **no depura el programa compilado**: corre un **intérprete tree-walking** propio (módulo bbk-debugger) que vuelve a parsear el .bbk y camina el AST. Es un **tercer motor de ejecución** en paralelo a los dos backends reales (bytecode JVM y C): lo que se depura no es lo que se ejecuta. Acoplar el debugger al programa compilado significa depurar el **bytecode real** que emite JvmCompiler, usando el mecanismo estándar de la plataforma Java: **JDI/JDWP** (la misma técnica que usan los debuggers de Java, Kotlin y Scala en IntelliJ). El programa se lanza en una **JVM aparte** con el agente de debug, y el IDE se **conecta** a ella para poner breakpoints, leer variables y hacer step.

Esto elimina el intérprete como motor, pero **traslada el costo** a tres lugares: el compilador tiene que emitir **más información de debug**, hay que **lanzar y orquestar un proceso externo** (hoy todo corre in-process dentro del IDE), y hay que **mapear los nombres y valores de la JVM de vuelta a BBK** (un emp.salary de BBK vive como campo emp\$salary, un INT como long, un decimal como BigDecimal). Es un cambio de arquitectura cohesivo, no una mejora incremental del intérprete actual.



1 Punto de partida (lo que hay hoy)

Pieza	Estado actual	Implicancia
Motor del debugger	bbk-debugger: intérprete tree-walking (Interpreter) que reparsea el .bbk y camina el AST, emitiendo un TraceStep por sentencia.	Se depura una segunda ejecución , no el bytecode real. Es el punto que el objetivo elimina.
Proceso de debug (plugin)	BbkDebugProcess corre el intérprete en un hilo del pool y se bloquea en una cola (resumeSignal) en cada pausa; mapea TraceStep → posición en el editor.	Toda la lógica de breakpoints/step/variables está atada al modelo del intérprete; se reescribe sobre JDI.
Ejecución (Run y Debug)	In-process: BbkProcessHandler → BbkRunner.compileAndRun carga el bytecode con un classloader propio y corre bbk.Main dentro del mismo JVM del IDE .	Para JDI hay que lanzar una JVM aparte : el agente JDWP no se conecta a la VM que ya corre el IDE.
Info de debug en el bytecode	LineNumberTable ya emitido (JvmCompiler.lineNumber() desde el SourceMap; test JvmDebugInfoTest verde).	Es la base y ya está . Falta el resto de la info de debug (abajo).

Pieza	Estado actual	Implicancia
Mapa de almacenamiento	Globales → campos <code>static</code> de <code>bbk.Main</code> ; subcampos de DS → campos <code>emp\$salary</code> (aplanados); locales → <code>slots</code> anónimos; INT→long, decimal→BigDecimal, CHAR→String.	JDI lee campos y slots, pero con nombres y tipos de la JVM : hay que revertirlos a la vista BBK.

2 Modelo objetivo (cómo funciona JDI)

JDI (com.sun.jdi, módulo `jdk.jdi` del JDK) es el cliente estándar de debug de la JVM — el mismo que usa el debugger de Java de IntelliJ. El IDE no «mete mano» dentro del programa: lanza una JVM con el agente JDWP, se conecta por un socket y le pide al agente que ponga breakpoints, frene, lea variables y haga step. La ventaja técnica concreta es que el **step** deja de ser conteo de profundidad casero y pasa a `StepRequest` nativo (INTO/OVER/OUT a nivel línea); los breakpoints, el call stack y las excepciones son los **reales** de la ejecución.

3 Pasos para acoplarlo

Esta es la secuencia de cambios concretos, en orden, de principio a fin. Es **un solo trabajo**: el debugger no funciona hasta que están todos. El paso 1 ya está hecho.

#	Paso	Dónde / cómo	Verificable sin ruido?
1	Números de línea (LineNumberTable)	<code>JvmCompiler.lineNumber()</code> ya los emite desde el <code>SourceMap</code> .	✓ Ya hecho test ASM ya verde
2	Atributo <code>SourceFile</code>	<code>cw.visitSource("Main.bbk", null)</code> — hoy va <code>null</code> . JDI necesita el nombre del fuente.	✓ Sí <code>ClassReader</code> assera el nombre
3	<code>LocalVariableTable</code>	Por cada local, <code>mv.visitLocalVariable(nombre, desc, null, start, end, slot)</code> . Hoy los slots son anónimos: el compilador conoce el nombre (la clave del mapa) pero lo descarta. Hay que llevar labels de rango vivo por método.	✓ Sí <code>ClassReader</code> assera la tabla
4	Escribir el <code>.class</code> a disco	El bytecode de <code>JvmCompiler</code> a un dir temporal (hoy vive en memoria, lo carga un classloader propio).	✓ Sí test headless de escritura
5	Forkear la JVM con el agente JDWP	<code>java -agentlib:jdwp=...,server=y,suspend=y -cp tmp bbk.Main</code> . Reemplaza la corrida in-process solo en modo Debug (Run sigue in-process).	✓ Sí se prueba por línea de comandos
6	Conectar por JDI	Nuevo módulo headless <code>bbk-jvm-debug</code> : <code>AttachingConnector</code> al socket + <code>ClassPrepareRequest</code> de <code>bbk.Main</code> .	✓ Sí JUnit: attach + carga
7	Mapear línea ↔ BBK	Reverso del <code>LineNumberTable</code> : <code>Location.lineNumber()</code> ↔ (archivo, línea) BBK, para instalar breakpoints y ubicar las pausas.	✓ Sí JUnit sobre la tabla

#	Paso	Dónde / cómo	Verificable sin runIde?
8	Instalar breakpoints	BreakpointRequest diferidos hasta el ClassPrepare, en las Location de las líneas mapeadas.	✓ Sí JUnit: frena donde toca
9	Event loop	Hilo que consume EventSets (breakpoint, step, muerte de la VM) y los traduce a positionReached.	✓ Sí JUnit: secuencia de eventos
10	Step nativo	StepRequest INTO/OVER/OUT a nivel línea — reemplaza el conteo de profundidad del intérprete.	✓ Sí JUnit: pasos esperados
11	Leer frames y variables	ThreadReference.frames(), StackFrame.visibleVariables() y los campos static de la clase.	✓ Sí JUnit: valores leídos
12	Mapear nombres y valores JVM → BBK	emp\$salary → emp.salary; reconstruir el DS como árbol; formatear long/BigDecimal/String/boolean estilo BBK.	● Parcial lógica testeable; la vista, runIde
13	Evaluador de expresiones	Evaluate / breakpoints condicionales / watches: reusar el parser de expresiones BBK y un mini-evaluador alimentado con los valores leídos del frame JDI.	✓ Sí JUnit del evaluador
14	Reescribir BbkDebugProcess sobre JDI	El XDebugProcess del plugin pasa a orquestar la sesión bbk - jvm-debug: breakpoints, frames, variables y step contra eventos JDI en vez del intérprete.	■ No solo runIde
15	Pipe de salida + ciclo de vida del proceso	Volcar stdout/stderr del proceso a la consola y manejar fin/kill (puerto JDWP, timeout, huérfanos — el problema que ya tuvimos con bootRun en :8080).	● Parcial parte headless, parte runIde
16	Cross-file: SMAP (JSR-45)	Todo compila a un solo bbk.Main con un SourceFile. Para depurar varios .bbk hay que emitir SourceDebugExtension (lo que hace Kotlin/JSP) o arrancar single-file. Decisión a tomar de entrada porque cambia el formato de debug del compilador.	● Parcial test del SMAP emitido

4 Qué se conserva y qué se retira

Se conserva	Se retira / reemplaza
• Toda la costura XDebugger del plugin (breakpoint type, editors provider, runner, consola). • El parser de expresiones BBK (lo reusa el evaluador). • El LineNumberTable ya emitido. • El BbkRunner in-process para el modo Run (no-debug).	• El Interpreter como motor de ejecución del debug (deja de existir el «tercer backend»). • El modelo DebugListener/TraceStep/resumeSignal (lo reemplazan los eventos JDI). • El BbkDebugProcess actual (se reescribe).

El Interpreter podría sobrevivir aparte como motor del comando **Trace BBK** (traza estática sin proceso), pero ya no como el debugger. Esa es una decisión a tomar.

5 Decisiones abiertas y riesgos

Tema	El punto
Cross-file (multi-archivo)	El intérprete junta todos los <code>.bbk</code> de la carpeta y depura cualquiera. Con un solo <code>bbk.Main</code> , JDI necesita SMAP/JSR-45 para mapear varias fuentes a una clase. Conviene decidir de entrada: implementar SMAP, o empezar por single-file y sumarlo después.
Proceso externo	Pasamos de in-process a forkear una JVM : hay que resolver el binario <code>java</code> , puerto JDWP, stdout, y matar huérfanos (el mismo tipo de problema que ya tuvimos con <code>bootRun</code> en <code>:8080</code>).
Granularidad del step	El intérprete frena limpio por sentencia. El bytecode frena por línea del <code>LineNumberTable</code> : expresiones compuestas o control de flujo pueden dar saltos menos «prolijos». Aceptable, pero distinto a lo de hoy.
Evaluación de expresiones	JDI puede <code>invokeMethod</code> , pero para expresiones BBK arbitrarias conviene evaluar del lado del plugin (parser BBK + valores leídos por JDI), no compilar cada expresión. Más simple y suficiente para watches/condicionales.
Dependencia <code>jdk.jdi</code>	<code>com.sun.jdi</code> viene en el JDK pero hay que ponerlo en el module path del nuevo módulo / plugin. Sin misterio, pero es wiring que hoy no existe.

6 Verificación

Buena parte es **testable headless** (sin abrir el IDE), por eso conviene meter la sesión JDI en un módulo aparte `bbk-jvm-debug`, igual que `bbk-debugger` es UI-agnóstico:

- **Headless (JUnit)**: que el compilador emita `SourceFile` + `LocalVariableTable` (+ SMAP) vía `ClassReader`; y una **sesión JDI mínima** — lanzar un programita, poner un breakpoint, verificar que frena y leer una variable. JDI corre sin GUI — son los pasos 1 a 13.
- **Solo runIde (lo verificás vos)**: el paso 14 (el `BbkDebugProcess` nuevo, el panel de Variables, el step con los botones, Evaluate). Esa parte **no la puedo verificar yo** — es la misma dependencia de siempre.

7 Recomendación

Sí vale acoplarlo al bytecode: es lo correcto (se depura lo que se ejecuta) y mata el tercer motor. El camino estándar es **JDI/JDWP con la sesión en un módulo headless** (`bbk-jvm-debug`) que el plugin orquesta — así el grueso queda **cubierto por tests** y solo la cáscara visual depende de `runIde`. Sobre cross-file conviene **decidir de entrada** si entra SMAP (multi-archivo) o arrancamos single-file, porque condiciona el formato de debug del compilador.

Es un cambio de arquitectura, no un parche: la secuencia recién da un debugger funcional cuando están todos los pasos. Por eso necesita ir **completo**, con tu verificación en `runIde` para el paso final de UI — no a medias.

Análisis grounded en el código actual: `JvmCompiler`, `BbkRunner`, `BbkDebugProcess`, `BbkProcessHandler` y el módulo `bbk-debugger`. Leyenda de «verificable»: ✓ headless ● **Parcial** ■ No (solo `runIde`).